

MedTech Meets Language Model; Transformers Healthcare

Abstract

The model is referenced from OpenAI GPT -2 paper called “Language Models for Unsupervised Multitask Learning” and OpenAI GPT -3 paper called “Language Models are Few Shot Learners”. Here, we make GPT-2 124M parameters model. The code snippet are Referenced from Andrej Karpathy Videos and Open source GitHub repositories.

Introduction

First of all, Everything started from the paper called “Attention is All you need”. This paper introduced us to Transformer Architecture, which was the key thing for all Artificial Intelligence development in this mid decade of 20s.

In this GPT -2, is a Decoder only Transformer. Here we adjust our GPT, by removing Encoder Block and its connective, called Multi Head Attention in the decoder Block. Here in this model, The OpenAI have added an Additional Layer Norm to it, before the MLP, and before the Softmax function. Here, the Layer Normalization are throughout the passway of model. Here, it follows the Single Convolutional stream from inputs through the decoder block and passthrough, the softmax probability. Also, it follows the block, contributes its Normalization.

Architecture Description

Block -> An Block class defines the architecture of the positional decoder of the transformer. The Block layers consists of layer norm 1, attention, i.e Casual Self Attention, layer norm 2, MLP (Multi Layer Perceptron or Feed Forward Network). The attention is Communication layer. Here, the x goes through, attention layer norm and it follows the MLP or Feed Forward Network via Layer norm. To understand, these are the ways, they gonna think and produce the logits(Probability).

MLP -> MLP(Multi Layer Perceptron) or Feedforward Network is layered with two Layer norm and between of Non Linear layernorm GELU(Gaussian Error Linear Unit). Gelu used to return the Distribution function of Gaussian Distribution. Due to Non Linearity Gradient issues, we use approximation of “Tanh” in gelu.

Causal Self Attention -> In previous GPT, the attention is used to manage the multi head i.e the actions are performed parallelly. This, Returns the concatenated of the actions that are parallelly happened. Tokens are lined up in three Vectors, i.e, Key, Query, Value (K, Q, V). The Key. Query are multiplied

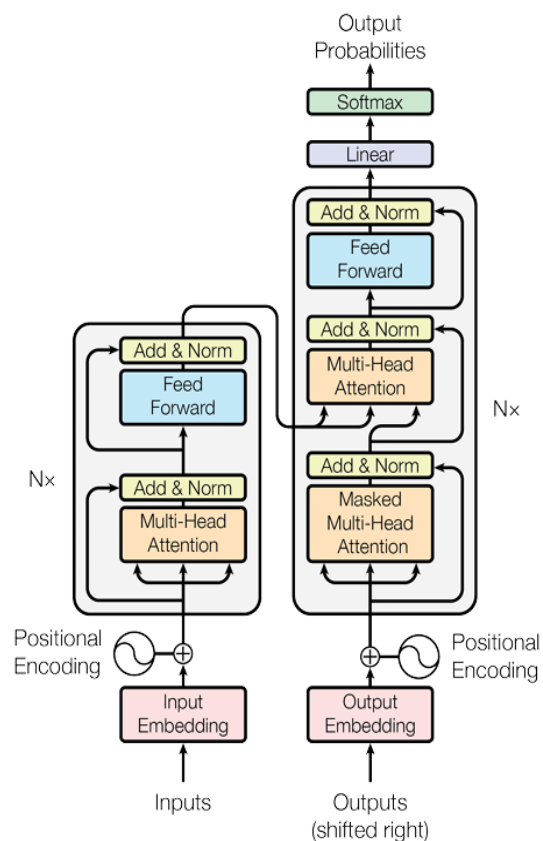


Figure 1: The Transformer - model architecture.

each other to get amount of attention. The attention is getted by multiplication of query and key transposed and taking sqrt of them. Then, they are masked fill with upper triangular matrix and applying softmax for normalisation. The weighted sum, that is returned value(y) is got by matrix multiplication of attention and value. Then, reassembling them, by contiguous view function. We use NANO_SCALE_GPT initialization for this module to scale this. Something interesting, instead of getting attention, masked fill them, then softmax, there is something called "FlashAttention". It is 7.6x faster than classic by Paper. It is implemented via Functional scaled dot product attention in torch.

GPT Initialization

Upto This part, we have completed the implementation of GPT 2 via transformer architecture. Here we initialize our Gpt model with hyper parameters, such as Block Size = 1024, Vocab Size = 50257, Number of layer = 12, number of head = 12, number of embedding = 768.. We make every number in nice even number for convention. Give it a try. Here, to notice that, Vocab Size is not a nice number, such that it is 50257. It refers, 50256 characters and last token is *End of Text* character, which makes it ugly. Here, End of text is introduced in GPT -2 Tokenization.

The Forward method to get the tokens. The Three Pillers, i.e. Batch size (B): How many sequences are processed in parallel. Sequence length (T): The number of tokens in each sequence. Model size (C): The hidden size determines the number of activations and parameters in the model. The indices forward pass to initialized transformer and return the logits.

Then, we import GPT-2 parameters from HuggingFace, that is trained by OpenAI by BPE(Byte Pair Encoding). Here, we imported GPT-2 (124M) parameter sized transformer for tokenization. The whole lives on GPU and are pretrained with the help of A100 GPU. The module "Tiktoken" is encoding token for GPT -2 is created by OpenAI. We take top 50 probabilities and Normalize it. Then, we take our model to GPU.. and certainly here, we need some dataset to train an model. For instance, Our model is a new born baby, certainly it need to learn vocabularies. We train them with "Tiny Shakespeare" dataset, which consists of most basic vocabularies. Just another, that we made the probabilities, i.e. logits, but the loss matters a lot. When a model is passed with input x, the error or overfit or underfit is called as its loss. Minimum the loss, Better the Accuracy of the Model. The loss is calculated via Functional Cross Entropy module by passing Batch times Token in 2D and the Vocab size to get the loss of the model. The Cross Entropy Loss at the initial stage of the Model Negative LogLikelihood of Vocab size.

Optimization

The Ultimate aim of Optimization is to reduce the training loss. We use AdamW(Adam with Weight Decay) optimizer to optimize our model. This AdamW optimizer is generally an Normalization that happens on each gradient and speeds up the optimization of the Language Models. It is that, optimization is always started with zero gradient. The learning rate of this AdamW model is, we fixed $3e-4$.

DataLoader is method to define certain batches that are to be used to trained an single epoch. We take current position B*T for batch time and run it on required times to achieve an single epoch.

Inconsistency:). Here we see, that lm_head(Language Modelling Head) and Transformer Word token Embedding Weight(transformer.wte.weight) have the same tensor. i.e they are pointing the same tensor, which lead to model inconsistency. To notice that, "Attention is All you need" paper, says "We share the same weight matrix between the two embedding layers and the pre softmax linear transformer". This can be resolved by Weight sharing. By assigning both to a same pointer, referring them multiples times as required.. the issue can be solved. Since 30% of the model is achieved at this state i.e ($768 * 50257 = 38M$) which nearly 30% of 124M. By making to points, we make the model more complex.. for space constraints, it is pointed same.

Initialising Weights, i.e, applying initial weights to the model with a method Initialising Weights,, by its type, i.e, Embedding or Linear.. In accordance to GPT -2 paper, the weight over layers are 768 – 1600, When taking normalization i.e, one over sqrt(N), we came across an value of 0.02 or nearby. This value 0.02 is fixed as Standard Deviation of the Model. We use manual seed for code reusability and reproducing the probabilities on all devices of Cuda. Upto this stage of implementation i.e, 50 iterations, on Tiny Shakespeare dataset, we achieve 6. Loss nearby. This works Yay!.

Making our money worth, We make some changes in our model to make it fast. To compile our model, yeah, torch has an option called Compile to make it fast. It will significantly reduce the time of compilation increases speedup. Speedup mainly comes from Python overhead and GPU read/writes. To say and single line of code, works 2.3x faster than previous.. Obviously checkout. Torch compile works in a way that, it sees all the lines of code, and makes an step overview of it. This significantly reduce the time. It just converts the python interpreter to compiler.

In Cuda, GPU, everypart, i.e, the compilation or multiplication are done via even numbers, especially they are in power of 2(Block Tiles), say that 2, 4, 8, 16, 64., etc, the vocabulary size 50257 is really ugly number, as it not in power of 2 or not an even number. So we round it off to 50304, as this is a nice number divisible by power of 2.

Learning Rate determines the magnitude of the changes to make to the weights and bias during each step of the gradient descent process. First we fix, it ((3e-4)), there is no clear consistency in GPT -2 paper or implementation in tensorflow. So we gonna consider, for instance, we take it from GPT -3 paper,, which is varying learning rate, i.e., Cosine decay Learning Rate. Initially, it reaches a maximum rate, then slowly decreasing from the maximum like Cosine function.

Here we, face an issue with our gpu, that is Batch Size!. For our tokens 1024, ($0.5e6 / 1024 = 488$, 0.5e6 refers 0.5M), we approximate fix batch size 488, which leads to explosion of GPU!. Here Gradient Descent comes into account. To overcome this, first fix an BatchSize. For example, if we take total batch size of 524288 (2^{19})(0.5M parameters), To calculate the gradient accumulation, ($\text{Total Batch Size} / (B * T)$) => ($2^{19} / (16 * 1024)$) == 32. To note that, total batch size must be divisible by (B*T). So from that, to achieve 0.5M parameter model, the optimizer has to pass forward and backward over the gradient accumulation times to achieve it.

GPU description

A100 80GB SXM – FP32 – 19.5 Tera Flops

Tensor Core – An Architecture in an A100 GPU, which basically does 4*4 matrix multiplication of the tensors. We use TensorFloat 32. In case of Tensorfloat 16, like gpu, there is an alternative called Automative Mixed Precision(AMP) i.e, torch autocast, which tries to reach accuracy of tensorfloat 32, and speeds up the training process.

Distributed Data Parallel

Time to bring heavy weapons! We here recommend to use Distributed Data Parallel, Not Data Parallel. A Single series of GPU contains 8 individual gpu. By using Torch run function, called Distributed Data Parallel, all the training can be distributed equally, it occurs parallely and efficiently.

Training Dataset

Open web text – Slim Pajama

Fine Web Edu – dataset

Open Source Medical datasets

Model Configuration

- **Block Size** (max sequence length): 512

- **Vocabulary Size:** 50,304
 - **Number of Layers:** 12
 - **Number of Heads:** 12
 - **Embedding Dimension:** 768
-

Parameters:

1. Embedding Layers:

- **Token Embedding (wte):**
Parameters=Vocab Size×Embedding Dimension=50,304×768=38,580,992
- **Positional Embedding (wpe):**
Parameters=Block Size×Embedding Dimension=512×768=393,216

2. Transformer Blocks: Each of the 12 blocks contains:

- **Attention Mechanism:**
 - **Query, Key, Value Projections (c_attn):**
Parameters=3×(Embedding Dimension×Embedding Dimension)=3×(768×768)=1,769,472
 - **Output Projection (c_proj):**
Parameters=Embedding Dimension×Embedding Dimension=768×768=589,824
 - **Total for Attention:** Parameters=1,769,472+589,824=2,359,296
- **Feedforward Network (MLP):**
 - **First Linear Layer (c_fc):**
Parameters=Embedding Dimension×(4×Embedding Dimension)=768×(4×768)=2,359,296
 - **Second Linear Layer (c_proj):**
Parameters=(4×Embedding Dimension)×Embedding Dimension=(4×768)×768=2,359,296
 - **Total for MLP:**
Parameters=2,359,296+2,359,296=4,718,592
- **Layer Normalization:** Each LayerNorm has parameters:
Parameters=2×Embedding Dimension=2×768=1,536
- **Total for two LayerNorms per block:** Parameters=2×1,536=3,072
- **Total per Block:**
Parameters=2,359,296(Attention)+4,718,592(MLP)+3,072(LayerNorms)=7,080,960
- **For 12 blocks:** Parameters=12×7,080,960=84,971,520

3. Final LayerNorm (ln_f):

Parameters=2×Embedding Dimension=2×768=1,536

4. Output Layer (lm_head):

- Shared with the token embedding layer (wte), so no additional parameters.

Total Parameters:

Total Parameters=Embedding Layers+Transformer Blocks+Final LayerNorm

=(38,580,992+393,216)+84,971,520+1,536

=123,947,264 (Approximately 124M)

Model Architecture:

```
GPT(
  (transformer): ModuleDict(
    (wte): Embedding(50304, 768)
    (wpe): Embedding(512, 768)
    (h): ModuleList(
      (0-11): 12 x Block(
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (attn): CausalSelfAttention(
          (c_attn): Linear(in_features=768, out_features=2304, bias=True)
          (c_proj): Linear(in_features=768, out_features=768, bias=True)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): MLP(
          (c_fc): Linear(in_features=768, out_features=3072, bias=True)
          (gelu): GELU(approximate='tanh')
          (c_proj): Linear(in_features=3072, out_features=768, bias=True)
        )
      )
    )
    (ln_f): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  )
  (lm_head): Linear(in_features=768, out_features=50304, bias=False)
)
```

Inference:

```
Generated Text:
what is Aortic: Aortic is a condition in which the aortic valve is so that it pumps blood out of the heart at greater than normal rate. When the aorta is too
large, the aorta can become blocked and the aortic valve cannot work properly.
- The use of the term to describe the type of food and the type of food that is consumed.
The study of the human body is a complex process and involves many different aspects. For example, the liver,
PS A:\Flutter\healthgt>
```

What is aortic valve?

Aortic valve is a valve that is attached to the heart valve. It is a valve that is attached to the heart valve. It is a valve that is attached to the heart valve. It is a valve that is attached to the heart valve. It is a valve that is attached